

HPC Final

CS 378 Programming for Correctness and Performance

Umer Khan, Ahmad Raeq Ahsen, Saood Usmani

April 30, 2025

1 Operation

Consider the operation

$$B := L^T B$$

where L^T is a $m \times m$ lower triangular matrix that will be transposed and behave like an upper triangle and B is a $m \times n$ matrix. This is a special case of matrix-matrix multiplication, with the Lower triangular matrix on the Left, We will refer to this operation as TRMM_LLTN where the LLTN stands for left lower transpose nonunit diagonal. The Nonunit diagonal means we will use the entries of the matrix that are stored on the diagonal. In some other cases, the entries on the diagonal may implicitly be taken to all equal one (Unit).

2 Precondition and Postcondition

In the precondition

$$B = \hat{B}$$

$\hat{B}$ denotes the original contents of B . This allows us to express the state upon completion, the postcondition, as

$$B = L^T \hat{B}.$$

It is implicitly assumed that L is nonunit lower triangular.

3 Partitioned Matrix Expressions and Loop Invariants

There are two PME's for this operation.

3.1 PME 1

To derive the first PME, partition

$$L \rightarrow \left(\begin{array}{c|c} L_{TL} & L_{BL}^T \\ \hline 0 & L_{BR} \end{array} \right), \quad \text{and} \quad B \rightarrow \left(\begin{array}{c} B_T \\ \hline B_B \end{array} \right).$$

Substituting these into the postcondition yields

$$\left(\begin{array}{c} B_T \\ \hline B_B \end{array} \right) = \left(\begin{array}{c|c} L_{TL} & L_{BL}^T \\ \hline 0 & L_{BR} \end{array} \right) \left(\begin{array}{c} \hat{B}_T \\ \hline \hat{B}_B \end{array} \right)$$

or, equivalently,

$$\left(\begin{array}{c} B_T \\ \hline B_B \end{array} \right) = \left(\begin{array}{c} L_{TL}\hat{B}_T + L_{BL}^T\hat{B}_B \\ \hline L_{BR}\hat{B}_B \end{array} \right),$$

which we refer to as the first PME for this operations.

From this, we can choose two loop invariants:

Invariant 1:

$$\left(\begin{array}{c} B_T \\ \hline B_B \end{array} \right) = \left(\begin{array}{c} L_{TR}\hat{B}_T \\ \hline \hat{B}_B \end{array} \right). \quad (1)$$

(The bottom part has been left alone and the top part has been partially computed).

Invariant 2:

$$\left(\begin{array}{c} B_T \\ \hline B_B \end{array} \right) = \left(\begin{array}{c} L_{TR}\hat{B}_T + (L_{BL}^T)^T\hat{B}_B \\ \hline \hat{B}_B \end{array} \right). \quad (2)$$

(The bottom part has been left alone and the top part has been completely computed).

3.2 PME 2

To derive the second PME, partition

$$B \rightarrow \left(B_L \mid B_R \right)$$

and do not partition L^T . Substituting these into the postcondition yields

$$\left(B_L \mid B_R \right) = L^T \left(\hat{B}_L \mid \hat{B}_R \right)$$

or, equivalently,

$$\left(B_L \mid B_R \right) = \left(L^T \hat{B}_L \mid L^T \hat{B}_R \right),$$

which we refer to as the second PME.

From this, we can choose two more loop invariants:

Invariant 3:

$$\left(B_L \mid B_R \right) = \left(L^T \widehat{B}_L \mid \widehat{B}_R \right). \quad (3)$$

(The left part has been completely finished and the right part has been left untouched).

Invariant 4:

$$\left(B_L \mid B_R \right) = \left(\widehat{B}_L \mid L^T \widehat{B}_R \right). \quad (4)$$

(The left part has been completely finished and the right part has been left untouched).

4 Deriving the Algorithms

4.1 Loop Invariant (5)

The unblocked algorithm for loop invariant (7) is showing in Figure 1. While the blocked algorithm for loop invariant (7) is showing in Figure 2.

Step	Algorithm: $[B] := \text{TRMM_LLTN_UNB_VAR1}(L, B)$
1a	$\{B = \hat{B}\}$
4	$B \rightarrow \left(B_L \mid B_R \right)$ where B_L has 0 columns
2	$\left\{ \left(B_L \mid B_R \right) = \left(L^T \hat{B}_L \mid \hat{B}_R \right) \right\}$
3	while $n(B_L) < n(B)$ do
2,3	$\left\{ \left(B_L \mid B_R \right) = \left(L^T \hat{B}_L \mid \hat{B}_R \right) \wedge n(B_L) < n(B) \right\}$
5a	$\left(B_L \mid B_R \right) \rightarrow \left(B_0 \mid b_1 \mid B_2 \right)$ where b_1 has 1 column
6	$\left\{ B = \left(L \hat{B}_0 \mid b_1 B_2 \right) \right\}$
8	$b_1 := L b_1$
7	$\left\{ B = \left(L \hat{B}_0 \mid L \hat{b}_1 \mid B_2 \right) \right\}$
5b	$B \rightarrow \left(B_L \mid B_R \right) \leftarrow \left(B_0 \mid b_1 \mid B_2 \right)$
2	$\left\{ \left(B_L \mid B_R \right) = \left(L^T \hat{B}_L \mid \hat{B}_R \right) \right\}$
	endwhile
2,3	$\left\{ \left(B_L \mid B_R \right) = \left(L^T \hat{B}_L \mid \hat{B}_R \right) \wedge \neg(n(B_L) < n(B)) \right\}$
1b	$\{[B] = \text{Trmm_lltn}(L, \hat{B})\}$

Figure 1: Unblocked Algorithm for Loop Invariant (5)

Step	Algorithm: $[B] := \text{TRMM_LLTN_BLK_VAR1}(L, B)$
1a	$\{B = \hat{B}\}$
4	$B \rightarrow \left(B_L \mid B_R \right)$ where B_L has 0 columns
2	$\left\{ \left(B_L \mid B_R \right) = \left(L^T \hat{B}_L \mid \hat{B}_R \right) \right\}$
3	while $n(B_L) < n(B)$ do
2,3	$\left\{ \left(B_L \mid B_R \right) = \left(L^T \hat{B}_L \mid \hat{B}_R \right) \wedge n(B_L) < n(B) \right\}$
5a	$\left(B_L \mid B_R \right) \rightarrow \left(B_0 \mid B_1 \mid B_2 \right)$ where B_1 has nc column
6	$\left\{ B = \left(L \hat{B}_0 \mid B_1 B_2 \right) \right\}$
8	$B_1 := L B_1$
7	$\left\{ B = \left(L \hat{B}_0 \mid L \hat{B}_1 \mid B_2 \right) \right\}$
5b	$B \rightarrow \left(B_L \mid B_R \right) \leftarrow \left(B_0 \mid B_1 \mid B_2 \right)$
2	$\left\{ \left(B_L \mid B_R \right) = \left(L^T \hat{B}_L \mid \hat{B}_R \right) \right\}$
	endwhile
2,3	$\left\{ \left(B_L \mid B_R \right) = \left(L^T \hat{B}_L \mid \hat{B}_R \right) \wedge \neg(n(B_L) < n(B)) \right\}$
1b	$\{[B] = \text{Trmm_lltn}(L, \hat{B})\}$

Figure 2: Blocked Algorithm for Loop Invariant (5)

5 Operation

Consider the operation

$$C := AA^T + C$$

where A is a $m \times k$ matrix and C is a $m \times m$ symmetric matrix, stored in its lower-triangular part. This is a special case of *symmetric rank- k update*, with the matrix A on the Left and its transpose A^T on the right (And by definition of symmetric, we know that AA^T is symmetric). We refer to this operation as SYRK_LN, where the LN stands for:

- left (the matrix A is on the left)
- no transpose (the matrix A is used as-is, without transposition)

This operation updates the symmetric matrix C using the product AA^T , storing the result only in the lower triangular part to exploit symmetry and save space.

6 Precondition and Postcondition

In the precondition

$$C = \hat{C}$$

$\hat{C}$ denotes the original contents of C . This allows us to express the state upon completion, the postcondition, as

$$C = AA^T + \hat{C}$$

7 Partitioned Matrix Expressions and Loop Invariants

7.1 PME 1

To derive the first PME, partition

$$A \rightarrow \left(\begin{array}{c} A_T \\ A_B \end{array} \right), \quad \text{and} \quad \hat{C} \rightarrow \left(\begin{array}{c|c} \hat{C}_{TL} & \hat{C}_{BL}^T \\ \hline \hat{C}_{BL} & \hat{C}_{BR} \end{array} \right).$$

Substituting into the postcondition yields

$$C = AA^T + \hat{C} \Rightarrow \left(\begin{array}{c|c} C_{TL} & C_{BL}^T \\ \hline C_{BL} & C_{BR} \end{array} \right) = \left(\begin{array}{c} A_T \\ A_B \end{array} \right) \left(\begin{array}{c} A_T \\ A_B \end{array} \right)^T + \left(\begin{array}{c|c} \hat{C}_{TL} & \hat{C}_{BL}^T \\ \hline \hat{C}_{BL} & \hat{C}_{BR} \end{array} \right)$$

Evaluating the multiplication yields:

$$\left(\begin{array}{c|c} C_{TL} & C_{BL}^T \\ \hline C_{BL} & C_{BR} \end{array} \right) = \left(\begin{array}{c|c} A_T A_T^T + \hat{C}_{TL} & A_T A_B^T + \hat{C}_{BL}^T \\ \hline A_B A_T^T + \hat{C}_{BL} & A_B A_B^T + \hat{C}_{BR} \end{array} \right)$$

This is the first PME for the operation.

Invariant 1:

$$\left(\begin{array}{c|c} C_{TL} & C_{BL}^T \\ \hline C_{BL} & C_{BR} \end{array} \right) = \left(\begin{array}{c|c} A_T A_T^T + \hat{C}_{TL} & \hat{C}_{BL}^T \\ \hline \hat{C}_{BL} & \hat{C}_{BR} \end{array} \right) \quad (5)$$

(Only the top-left block has been updated.)

Invariant 2:

$$\left(\begin{array}{c|c} C_{TL} & C_{BL}^T \\ \hline C_{BL} & C_{BR} \end{array} \right) = \left(\begin{array}{c|c} A_T A_T^T + \hat{C}_{TL} & A_T A_B^T + \hat{C}_{BL}^T \\ \hline A_B A_T^T + \hat{C}_{BL} & \hat{C}_{BR} \end{array} \right) \quad (6)$$

(The entire top row and left column have been fully updated.)

Invariant 3:

$$\left(\begin{array}{c|c} C_{TL} & C_{BL}^T \\ \hline C_{BL} & C_{BR} \end{array} \right) = \left(\begin{array}{c|c} \hat{C}_{TL} & \hat{C}_{BL}^T \\ \hline \hat{C}_{BL} & A_B A_B^T + \hat{C}_{BR} \end{array} \right) \quad (7)$$

(Only the bottom-right block has been updated.)

Invariant 4:

$$\left(\begin{array}{c|c} C_{TL} & C_{BL}^T \\ \hline C_{BL} & C_{BR} \end{array} \right) = \left(\begin{array}{c|c} \hat{C}_{TL} & A_T A_B^T + \hat{C}_{BL}^T \\ \hline A_B A_T^T + \hat{C}_{BL} & A_B A_B^T + \hat{C}_{BR} \end{array} \right) \quad (8)$$

(The entire bottom row and right column have been updated, leaving the top-left untouched.)

7.2 PME 2

To derive the second PME, partition

$$A \rightarrow \left(A_L \mid A_R \right)$$

and do not partition $\hat{C}$.

Substituting this into the postcondition yields

$$C = A A^T + \hat{C} \Rightarrow C = \left(A_L \mid A_R \right) \begin{pmatrix} A_L^T \\ A_R^T \end{pmatrix} + \hat{C}$$

Evaluating the matrix product gives:

$$C = A_L A_L^T + A_R A_R^T + \hat{C}$$

This expression is referred to as the second PME for this operation. From this, we can choose two more loop invariants:

Invariant 5:

$$C = A_L A_L^T + \widehat{C} \quad (9)$$

(All of C is being partially computed on A moves from left to right).

Invariant 6:

$$C = A_R A_R^T + \widehat{C} \quad (10)$$

(All of C is being partially computed on A moves from right to left).

8 Deriving the Algorithms

8.1 Loop Invariant (5)

The unblocked algorithm for loop invariant (5) is showing in Figure 3. While the blocked algorithm for loop invariant (5) is showing in Figure 4.

Step	Algorithm: $[C] := \text{SYRK_LN_UNB_VAR1}(A, A, C)$
1a	$\{C = \widehat{C}$
4	$A \rightarrow \left(A_L \middle A_R \right), A \rightarrow \left(A_L \middle A_R \right)$ where A_L has 0 columns, A_L^T has 0 columns
2	$\{C = A_L A_L^T + \widehat{C}$
3	while $n(A_L) < n(A)$ do
2,3	$\left\{ \begin{array}{l} C = A_L A_L^T + \widehat{C} \wedge n(A_L) < n(A) \end{array} \right\}$
5a	$\left(A_L \middle A_R \right) \rightarrow \left(A_0 \middle a_1 \ A_2 \right), \left(A_L^T \middle A_R^T \right) \rightarrow \left(A_0^T \middle a_1^T \ A_2^T \right)$ where a_1 has 1 column, a_1^T has 1 column
6	$\left\{ \begin{array}{l} C = A_0 A_0^T + \widehat{C} \end{array} \right\}$
8	$C := a_1 (a_1^T)^T + C$
7	$\left\{ \begin{array}{l} C = \left(A_L \ a_1 \right) \left(A_L^T \ a_1^T \right) + \widehat{C} \end{array} \right\}$
5b	$A \rightarrow \left(A_L \middle A_R \right) \leftarrow \left(A_0 \ a_1 \middle A_2 \right), A \rightarrow \left(A_L^T \middle A_R^T \right) \leftarrow$ $\left(A_0^T \ a_1^T \middle A_2^T \right)$
2	$\left\{ \begin{array}{l} C = A_L A_L^T + \widehat{C} \end{array} \right\}$
	endwhile
2,3	$\{C = A_L A_L^T + \widehat{C} \wedge \neg(n(A_L) < n(A))\}$
1b	$\{[C] = \text{Syrk_ln}(A, A, \widehat{C})\}$

Figure 3: Blocked Algorithm for Loop Invariant (5)

Step	Algorithm: $[C] := \text{SYRK_LN_BLK_VAR1}(A, A, C)$
1a	$\{C = \widehat{C}$
4	$A \rightarrow \left(A_L \middle A_R \right), A \rightarrow \left(A_L \middle A_R \right)$ where A_L has 0 columns, A_L has 0 columns
2	$\{C = A_L A_L^T + \widehat{C}$
3	while $n(A_L) < n(A)$ do
2,3	$\left\{ \begin{array}{l} C = A_L A_L^T + \widehat{C} \wedge n(A_L) < n(A) \end{array} \right\}$
5a	$\left(A_L \middle A_R \right) \rightarrow \left(A_0 \middle A_1 \ A_2 \right), \left(A_L^T \middle A_R^T \right) \rightarrow \left(A_0^T \middle A_1^T \ A_2^T \right)$ where A_1 has b columns, A_1^T has b columns
6	$\left\{ \begin{array}{l} C = A_0 A_0^T + \widehat{C} \end{array} \right\}$
8	$C := A_1 (A_1^T)^T + C$
7	$\left\{ \begin{array}{l} C = \left(A_L \middle A_1 \right) \left(A_L^T \middle A_1^T \right) + \widehat{C} \end{array} \right\}$
5b	$A \rightarrow \left(A_L \middle A_R \right) \leftarrow \left(A_0 \ A_1 \middle A_2 \right), A \rightarrow \left(A_L^T \middle A_R^T \right) \leftarrow$ $\left(A_0^T \ A_1^T \middle A_2^T \right)$
2	$\left\{ \begin{array}{l} C = A_L A_L^T + \widehat{C} \end{array} \right\}$
	endwhile
2,3	$\{C = A_L A_L^T + \widehat{C} \wedge \neg(n(A_L) < n(A))\}$
1b	$\{[C] = \text{Syrk_ln}(A, A, \widehat{C})\}$

Figure 4: Blocked Algorithm for Loop Invariant (5)